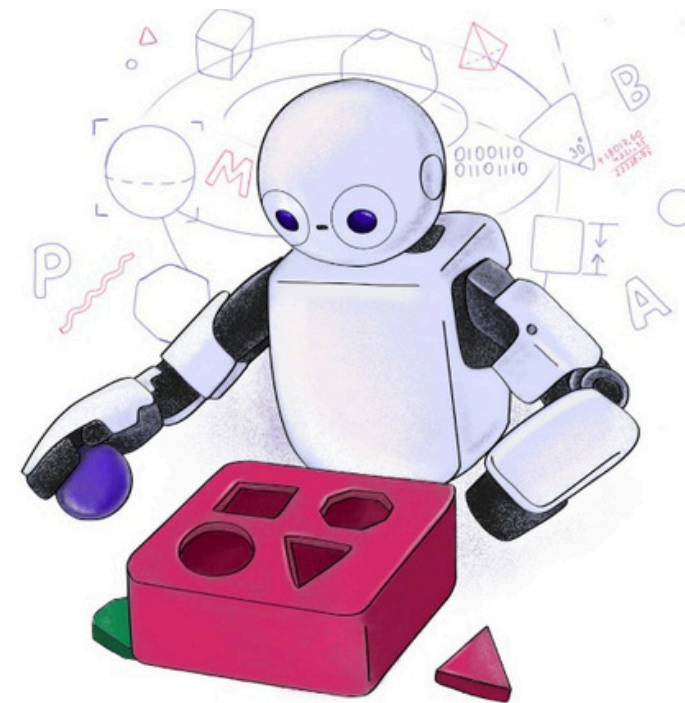


TP558 - Tópicos avançados em Machine Learning:

***TinyWolf: Efficient on-device TinyML training for
IoT using enhanced Grey Wolf Optimization***



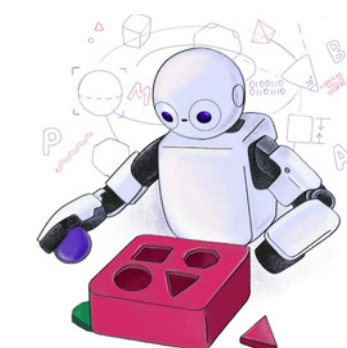
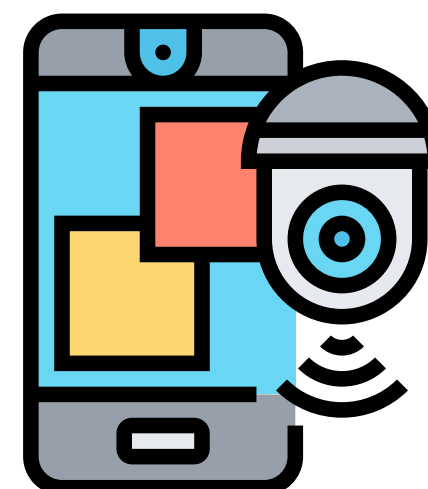
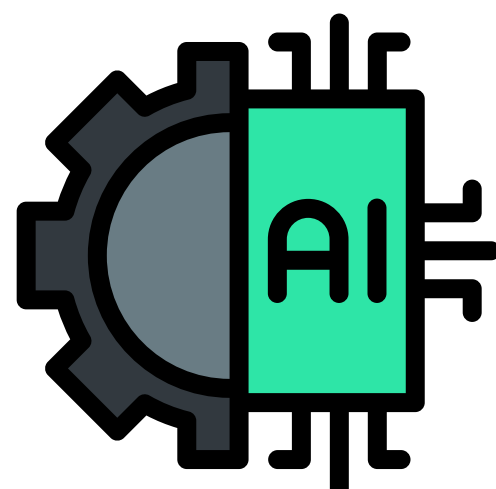
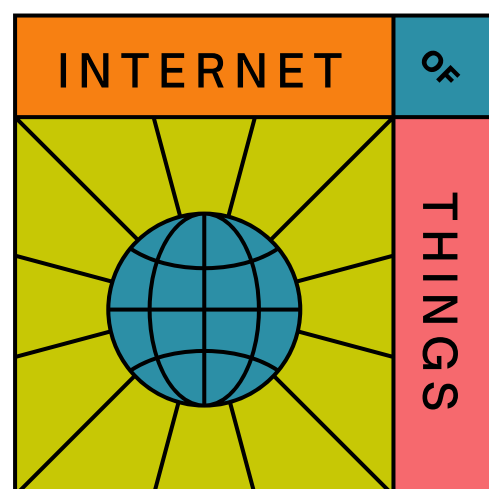
Inatel

Rafaella Laureano Dias
rafaella.dias@mtel.inatel.br

Introdução



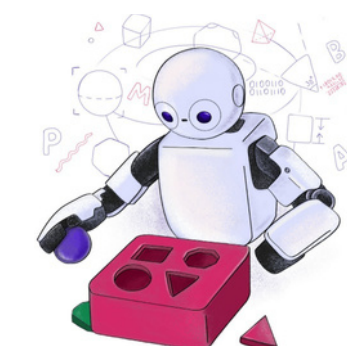
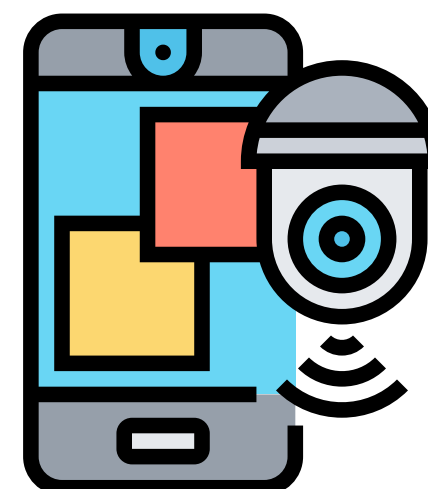
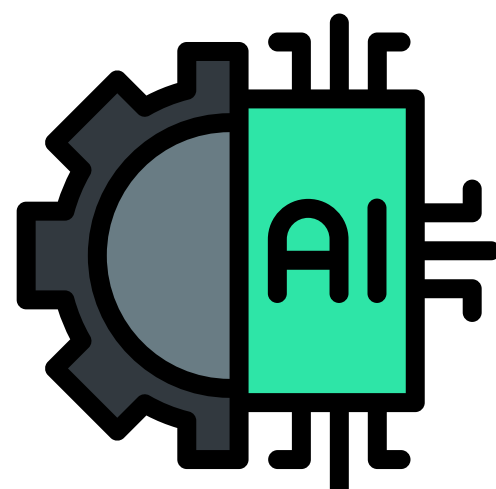
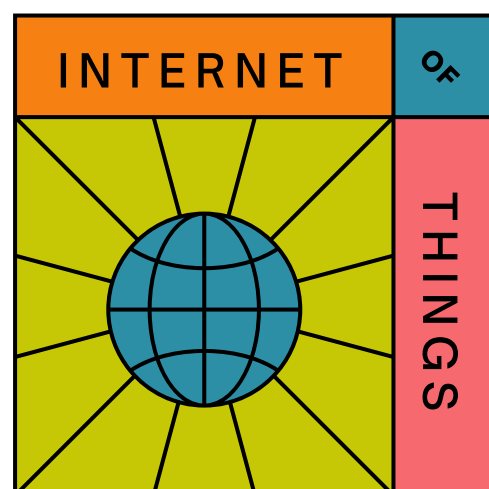
Nas últimas décadas, as tecnologias embarcadas passaram por uma verdadeira revolução, com o surgimento de pequenos sistemas eletrônicos compostos por microcontroladores, sensores e atuadores capazes de executar tarefas específicas. Esses sistemas tornaram-se a base da Internet das Coisas (IoT – Internet of Things), cujo objetivo é conectar objetos do mundo físico à internet, permitindo a coleta e o compartilhamento contínuo de dados. No entanto, essa arquitetura apresenta um problema estrutural: a maioria dos dispositivos IoT ainda não é inteligente. Em geral, eles apenas capturam informações e as enviam para a nuvem (cloud computing), onde modelos de aprendizado de máquina processam os dados e retornam os resultados. Esse modelo, centrado na nuvem (cloud-centric), traz limitações significativas, como alta latência, riscos à privacidade e segurança, além da dependência total de conectividade — se a internet falha, todo o sistema é comprometido. Para mitigar esses desafios, surgiram novas camadas de processamento próximas à borda da rede, como o fog computing, edge computing e mobile edge computing. Contudo, mesmo com essas evoluções, a inteligência ainda permanecia fora do dispositivo, mantendo os sensores como elementos essencialmente “cegos e burros”.



Introdução



O TinyML (Tiny Machine Learning) surge como um novo paradigma que leva a inteligência artificial diretamente para dentro do microcontrolador (MCU) — o mesmo chip responsável por ler sensores, controlar atuadores e operar com baterias extremamente limitadas. A proposta central é embarcar o aprendizado de máquina no próprio dispositivo, permitindo que ele analise localmente os sinais capturados, tome decisões autônomas (como acionar alertas ou motores) e envie para a nuvem apenas as informações realmente relevantes. Dessa forma, o TinyML combina duas grandes revoluções tecnológicas: o avanço dos algoritmos de aprendizado de máquina (como CNNs, RNNs e LSTMs) e a miniaturização de hardware embarcado de baixíssimo consumo energético. O resultado é o surgimento de uma nova geração de dispositivos inteligentes na borda (smart edge devices), capazes de processar e raciocinar sem depender constantemente da nuvem.

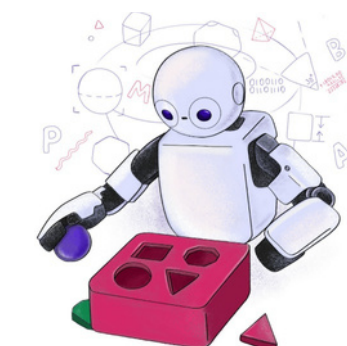
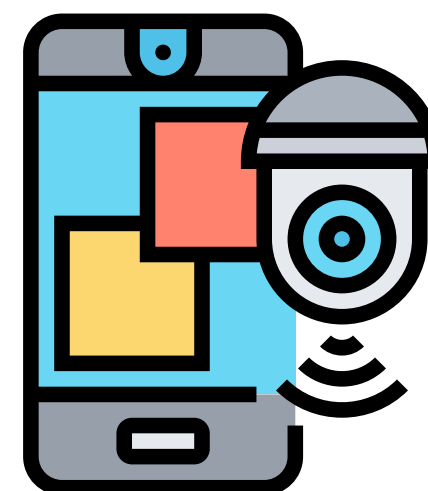
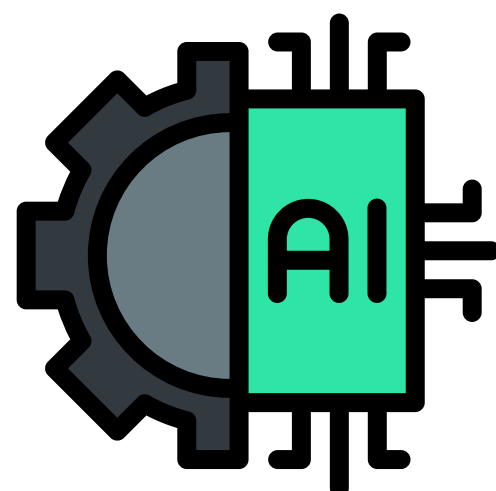
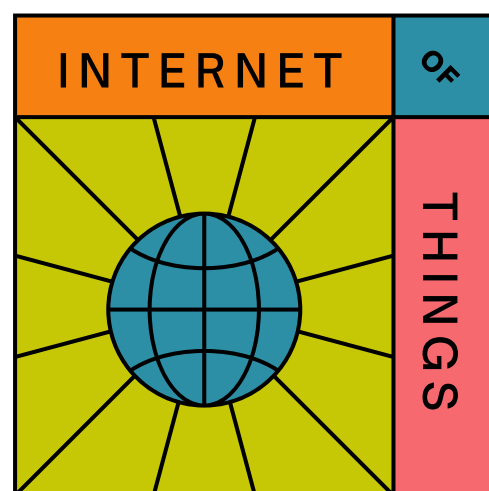


Introdução

É nesse contexto que o artigo TinyWolf se baseia. Ele aborda o desafio de treinar redes neurais diretamente dentro de microcontroladores TinyML, que possuem memória extremamente restrita. Os autores demonstram que, embora otimizadores baseados em gradientes (como SGD e Adam) ofereçam alta acurácia, seu consumo de memória inviabiliza o treinamento on-device. Como alternativa, propõem o uso de meta-heurísticas inspiradas na natureza — em especial o Grey Wolf Optimizer (GWO) — adaptadas para um gerenciamento mais eficiente de memória SRAM e paging/swap em flash entre as hierarquias dos lobos (α , β , δ , ω). Essa abordagem resultou em até 71% de economia de memória em comparação com métodos tradicionais, viabilizando o treinamento direto em MCUs com apenas 256 KB de RAM, validado em 13 conjuntos de dados públicos.

Entre as principais contribuições do estudo estão:

- A comparação abrangente entre otimizadores de gradiente e de enxame, buscando o de menor uso de memória;
- A otimização do GWO para reduzir ainda mais o consumo de SRAM;
- E uma análise detalhada de hiperparâmetros do método proposto, consolidada em benchmarks com 13 datasets de diferentes domínios.



Fundamentação teórica



As principais vantagens, conhecidas como Key Performance Indicators (KPIs), que demonstram o impacto prático dessa tecnologia:

De “burro” a inteligente – Sensores passam de simples coletores de dados para dispositivos capazes de interpretar o que medem.
Exemplo: um sensor de vibração em uma turbina identifica padrões que indicam falha iminente, alertando o sistema antes da quebra.

Menor uso de banda e rede – Ao processar localmente, reduz-se o volume de dados transmitidos.
Exemplo: um Boeing 787 gera 5 GB/s e um carro autônomo 1 GB/s — inviável enviar tudo à nuvem.

Segurança e privacidade – Os dados permanecem no dispositivo, reduzindo riscos de interceptação, vazamento ou uso indevido.

Redução da latência – Em aplicações críticas, como cirurgias robóticas ou veículos autônomos, decisões em milissegundos são vitais.

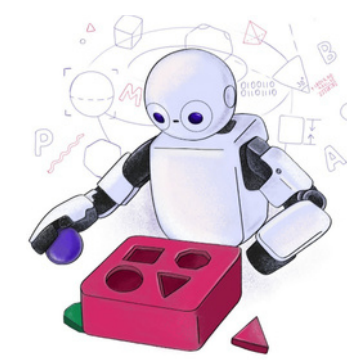
Confiabilidade – Mesmo sem conexão à internet, o sistema continua operando, essencial em ambientes isolados (fazendas, navios, montanhas).

Eficiência energética – Processar localmente consome muito menos energia do que transmitir continuamente os dados.

Resiliência a falhas – A inteligência distribuída evita que o sistema colapse caso a nuvem fique indisponível.

Filtragem de dados – O dispositivo pode eliminar redundâncias.
Exemplo: uma câmera de segurança descarta 99 % das imagens sem movimento.

Baixo custo operacional – Menos transmissão implica menor uso de rede, armazenamento e energia, reduzindo o custo total da solução.

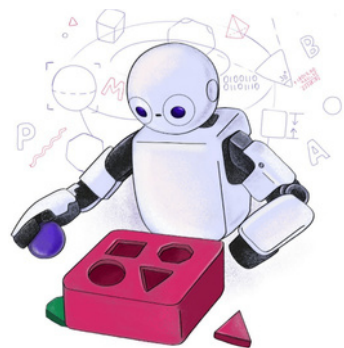


Fundamentação teórica



A tabela a seguir destaca as principais diferenças entre o Machine Learning (ML) Tradicional e o TinyML, evidenciando o contraste entre a computação em nuvem e a inteligência embarcada:

Característica	ML Tradicional	TinyML
Local de execução	Nuvem, GPU, Data Center	Microcontrolador (MCU)
Memória disponível	De GBs a TBs	256 KB a 1 MB
Energia	Ilimitada (energia elétrica)	Limitada (bateria)
Latência	Alta (rede + processamento)	Quase zero
Privacidade	Dados saem do dispositivo	Dados permanecem locais
Aplicações típicas	Visão computacional em larga escala, tradução automática	Sensores, wearables, IoT rural, automação doméstica



Fundamentação teórica



Como “encaixar” o Machine Learning dentro de um microcontrolador?

a) Knowledge Distillation (Destilação de Conhecimento)

Essa técnica consiste em ensinar um modelo menor (student) a reproduzir o comportamento de um modelo maior e mais poderoso (teacher).

O modelo teacher — por exemplo, uma CNN completa treinada em GPU — gera “saídas suaves” (soft targets), que indicam não apenas a classe correta, mas também as probabilidades das demais classes. O modelo student aprende a imitar essas respostas, capturando o raciocínio do teacher com muito menos parâmetros.

👉 Resultado: grande redução de tamanho do modelo, mantendo boa precisão.

b) Pruning (Poda de Conexões)

Após o treinamento, são removidas as conexões e pesos menos relevantes, reduzindo drasticamente a complexidade do modelo.

O processo é iterativo: treinar → remover → reentreinar → repetir.

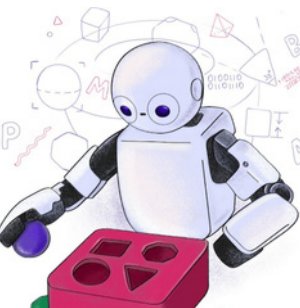
Com isso, até 90% dos parâmetros podem ser eliminados, resultando em modelos esparsos, leves e muito mais rápidos para execução embarcada.

c) Quantization (Quantização)

Substitui representações numéricas de alta precisão (float32) por inteiros de 8 bits (int8).

Essa conversão reduz o tamanho dos pesos em até 75% e acelera as operações matemáticas, mantendo uma perda de acurácia mínima — geralmente entre 1% e 3%.

👉 É uma das técnicas mais usadas em deploys TinyML.



Fundamentação teórica

d) Encoding (Codificação)

Aplica métodos de compressão sem perdas, como o Huffman Coding, para representar pesos que aparecem com frequência usando menos bits.

Esse processo reduz ainda mais o tamanho final do modelo — podendo alcançar até 49× de compactação em alguns casos.

e) Compilation (Compilação)

Na etapa final, o modelo compactado é convertido em código C/C++ para ser executado diretamente no microcontrolador.

Ferramentas como o TensorFlow Lite Micro (TFLM) automatizam essa conversão, gerando um código otimizado e embarcável que permite a inferência em tempo real dentro de MCUs com poucos kilobytes de RAM.

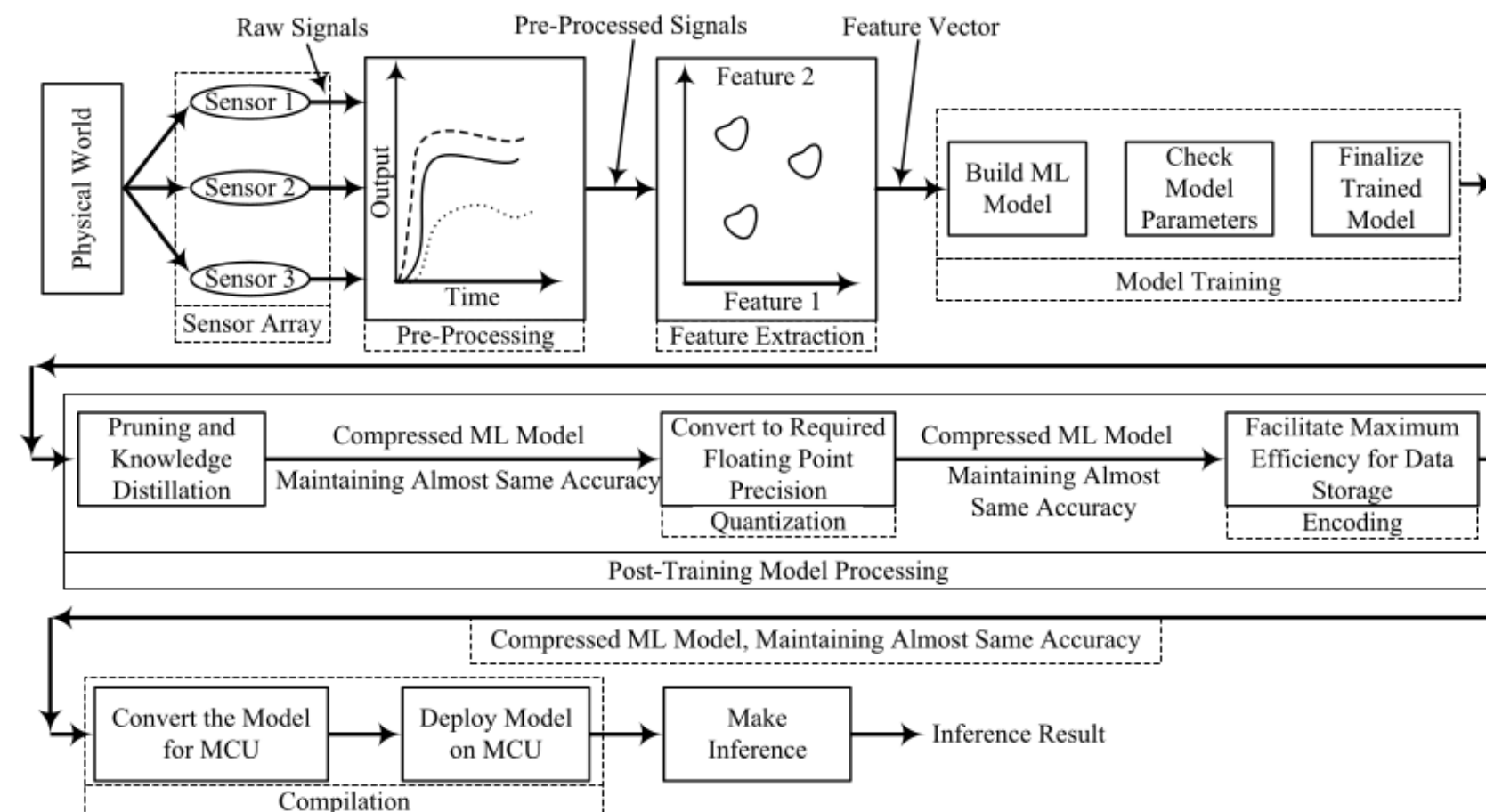
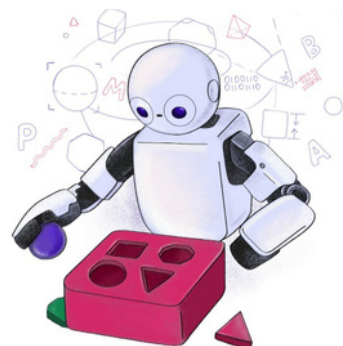


Fig. 2. TinyML architecture.

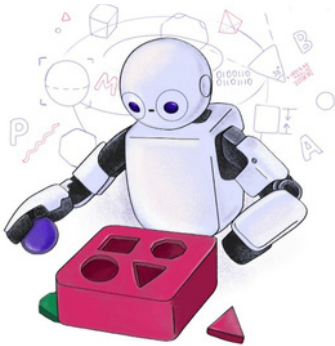


Fundamentação teórica

Inatel

Essas ferramentas tornam viável o desenvolvimento TinyML mesmo para engenheiros sem profundo conhecimento em redes neurais.

Ferramenta	Desenvolvedor	Destaques
TensorFlow Lite Micro (TFLM)	Google	Mais difundido; suporte amplo a MCUs (Arduino, STM32).
Embedded Learning Library (ELL)	Microsoft	Conversor de modelos e compilador cruzado.
GLOW	Facebook	Compilador para aritmética quantizada.
STM32 X-CUBE.AI	STMicroelectronics	Gera código otimizado para chips STM32.
TinyEngine	Academia	Otimiza memória e inferência.
TVM	Apache	Compilador universal de redes neurais.
uTensor	ARM	Conversão direta de modelos TensorFlow em C++ para Cortex-M.



Arquitetura e funcionamento

(Artigo TinyWolf)

Inatel



Conjuntos de Dados Utilizados

Os autores utilizaram 13 conjuntos amplamente disponíveis (Zenodo) com tarefas de classificação binária e multiclasse: Heart Attack, Diabetes, Kidney Stone, Breast Cancer (UCI FNA), BMI Index, Titanic, Fish Species, Wheat Seed, Red Wine (Vinho Verde), Customer Behavior, Sonar (Rock vs Mine), Loan Prediction e Airline Passenger Satisfaction.



Especificações do Sistema

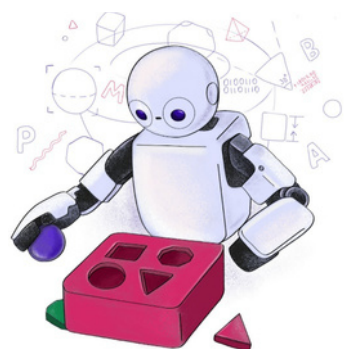
Servidor (PC): Intel i5-10300H, 16 GB DDR4, GPU GTX 1650 Ti (4 GB), armazenamento NVMe e Linux 5.11.
Microcontrolador: nRF52840 (Cortex-M4F, 64 MHz), com 1 MB de flash e 256 KB de SRAM, além de memória NOR flash externa W25Q32JVSSIQ (32 Mb, 133 MHz) e comunicação Bluetooth 2.4 GHz com o PC.



Arquitetura da Rede Neural Base

A rede foi projetada para ser pequena e eficiente, adequada ao contexto embarcado:

- 1ª camada densa: 6 neurônios (ReLU)
- 2ª camada densa: 20 neurônios (ReLU)
- Camada final: ajustada ao número de classes, com Softmax
- A função de perda é escolhida conforme o problema:
- Binária: binary cross-entropy
- Multiclasse: categorical cross-entropy



Arquitetura e funcionamento

(Artigo TinyWolf)

Inatel

✚ Framework de Teste TinyML (Simulação → Embarcado)

O fluxo experimental envolve:

- Simulação no PC para estimar o consumo de memória de cada otimizador.
- Seleção dos otimizadores que cabem no orçamento de memória.
- Portabilidade do mais eficiente — o GWO (TinyWolf) — para o MCU.
- Uso da flash como área de swap/paging, compensando a limitação de RAM.
- Otimizações para Caber no MCU

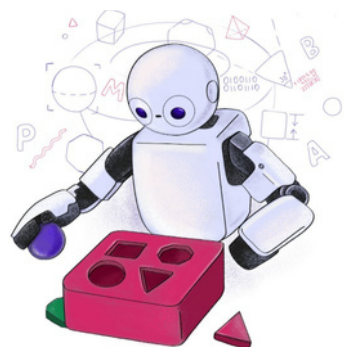
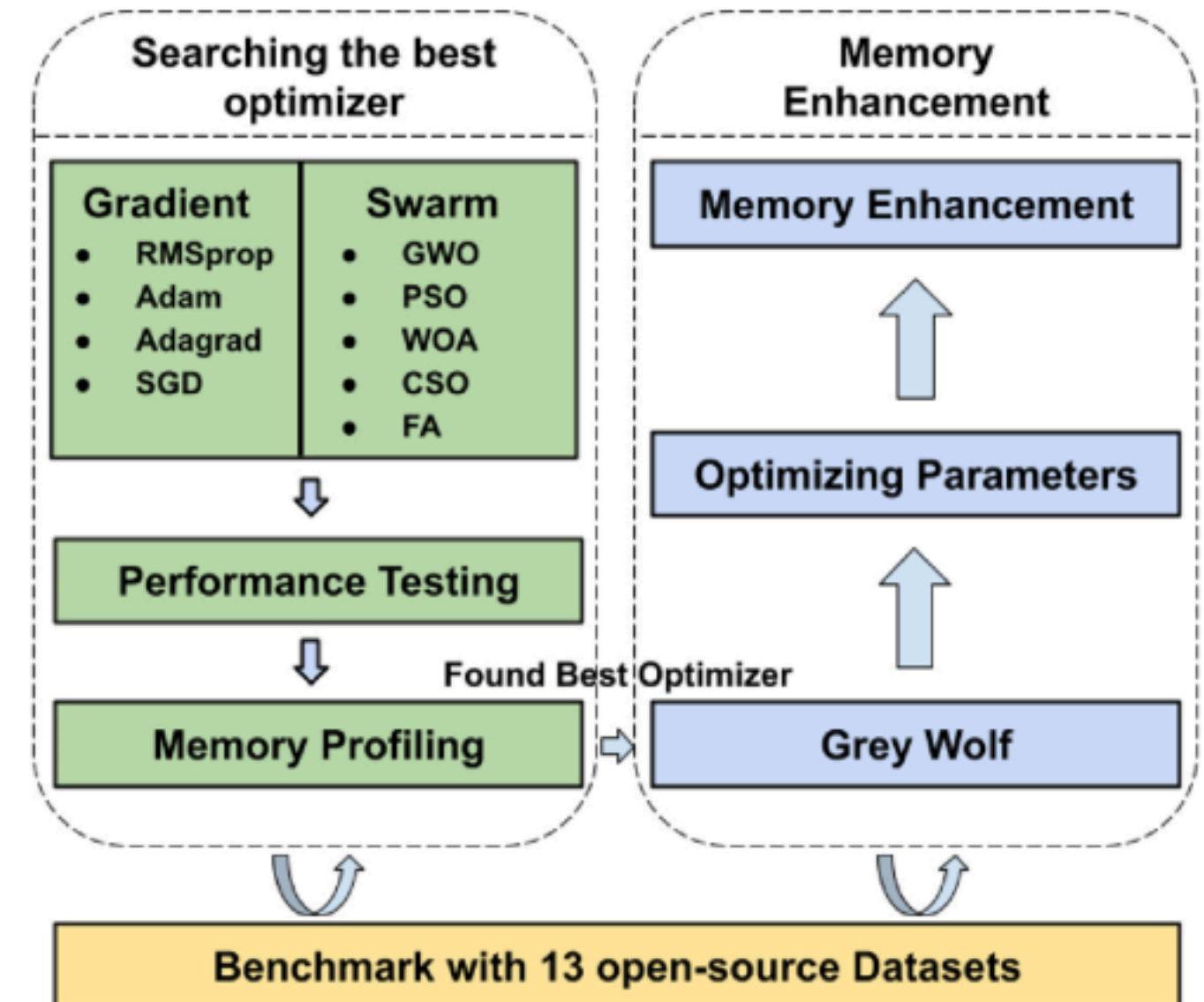
Para viabilizar o treinamento:

- Aproximações rápidas de funções não lineares, como a sigmóide aproximada por

$$f(x) = x / (1 + |x|)$$

ou lookup tables com interpolação linear — reduzindo o custo computacional.

- Poda iterativa de pesos — elimina conexões com variação insignificante
- ($\Delta\epsilon \rightarrow 0$) — reduzindo a matriz de pesos até caber no MCU.



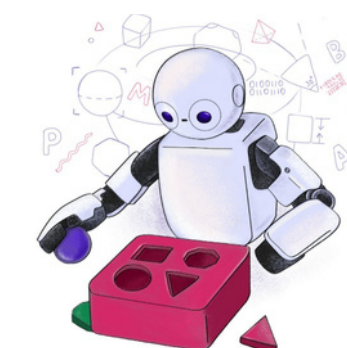
Arquitetura e funcionamento

(Para lembrar otimizadores)

Inatel

Os **otimizadores de gradiente** são métodos clássicos usados para treinar redes neurais, ajustando os pesos do modelo para minimizar o erro (função de perda). Eles funcionam calculando o gradiente — ou seja, a direção de maior variação da perda — e atualizando os parâmetros para o sentido oposto, buscando o ponto ótimo.

- SGD (Stochastic Gradient Descent): Atualiza os pesos com base em amostras aleatórias do conjunto de dados, tornando o processo rápido e adequado para grandes volumes de dados. Porém, pode gerar oscilações por ser estocástico.
- RMSProp: Adapta a taxa de aprendizado de cada parâmetro individualmente, levando em conta a média dos gradientes anteriores. É mais estável e converge mais rápido, especialmente em funções complexas.
- Adam (Adaptive Moment Estimation): Combina o conceito de momentum (para acelerar o aprendizado) com a adaptação de taxas individuais, resultando em um dos otimizadores mais eficientes e populares em redes profundas.
- Adagrad: Ajusta automaticamente a taxa de aprendizado para cada parâmetro, favorecendo características raras. Funciona bem em dados esparsos, mas tende a reduzir demais a taxa de aprendizado ao longo do tempo.



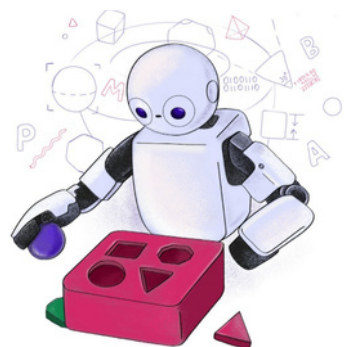
Arquitetura e funcionamento

(Para lembrar otimizadores)

Inatel

Os **otimizadores de enxame (Swarm)** são inspirados no comportamento coletivo de animais como lobos, pássaros, peixes e insetos. Eles não usam gradientes, mas buscam a melhor solução por meio de cooperação e competição entre indivíduos de um grupo (a “população” de soluções). São ideais para ambientes com poucos recursos e problemas complexos.

- Grey Wolf Optimization (GWO): Baseado na hierarquia social dos lobos (α , β , δ e ω). Os melhores indivíduos “guiam” os demais na busca pela solução ótima, equilibrando exploração e refinamento.
- Particle Swarm Optimization (PSO): Inspirado em bandos de pássaros e cardumes de peixes, onde cada partícula aprende com sua experiência e com a melhor do grupo, atualizando sua posição no espaço de soluções.
- Whale Optimization Algorithm (WOA): Simula o comportamento de caça das baleias, que cercam e convergem sobre a presa — útil para ajustar continuamente soluções promissoras.
- Cuckoo Search (CSO): Inspirado nos cucos, que colocam ovos em ninhos de outras aves. Cada “ovo” representa uma solução, e as piores são substituídas por novas candidatas, promovendo diversidade.
- Firefly Algorithm (FA): Baseado na atração entre vaga-lumes por meio da luz. Soluções mais “brilhantes” (melhores) atraem as demais, orientando o processo de otimização.



Arquitetura e funcionamento

O GWO mantém uma “alcateia” de soluções (vetores de pesos). No TinyWolf:

- α , β , δ (as 3 melhores soluções) ficam na SRAM — acesso rápido.
- As demais ω (soluções “restantes”) ficam na flash (swap/paging) — acesso eventual.
- Esse “swap inteligente” reduz muito a pressão de RAM durante a exploração/exploração do espaço de soluções, permitindo treinar no próprio MCU.

Hiperparâmetros típicos que os autores usam:

- Tamanhos da alcateia (número de “lobos”): 10, 50 ou 200,
- Até 100 iterações,
- Limites dos parâmetros em $[-1,1]$,
- Dimensão = n° de pesos + biases da rede

Start

Step 1: Let, $w \leftarrow W_{olf}$.

Step 2: Let, $n = 50w$.

Step 3: For $i = 0$ to n :

$w \leftarrow \text{random weights}$

Step 4: Let, $f_i \leftarrow \text{fitness}(w_i)$

$f \leftarrow \text{sort}(f)$

$\alpha \leftarrow f(0)$, $\beta \leftarrow f(1)$, $\delta \leftarrow f(2)$ and $\omega \leftarrow f(3 \text{ to } n - 1)$

Step 5: SRAM $\leftarrow \alpha$, β and δ .

Swap Space $\leftarrow \omega$.

Step 6: Use α , β and δ to search solution space.

Step 7: SRAM $\leftarrow \omega$

ω updated according to α , β and δ

Step 8: Step 4 to 7 are iterated till convergence.

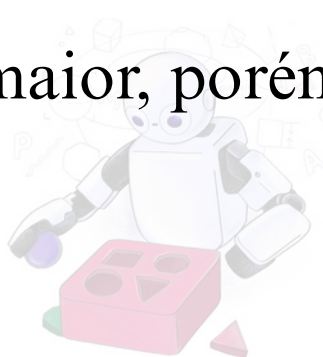
Step 9: After convergence, $\alpha == \text{solution}$

Step 10: $\text{TinyML_model_weights} \leftarrow \alpha$.

End

PS: SRAM (Static Random Access Memory) é a memória principal do microcontrolador, ela é muito rápida, usada para guardar os dados e variáveis ativas enquanto o programa está rodando, mas é pequena e limitada. A flash é uma memória não volátil, ela continua armazenando dados mesmo quando o dispositivo é desligado, ela é maior, porém, mais lenta.

“Swap” e “paging” são técnicas para usar a memória flash como se fosse uma “extensão” da SRAM.



Arquitetura e funcionamento

A imagem ao lado mostra como o TinyWolf atualiza os pesos no MCU:

- **Inicialização:**

Gera muitos candidatos de pesos ω (a “alcateia”). Avalia cada uma com a função de perda. Ordena. Define α (melhor), β e δ . Coloque α , β , δ na SRAM e ω na swap.

- **Busca/atualização:**

Usa α , β , δ para guiar a atualização das soluções ω (movimento de caça do GWO). Traz ω para SRAM quando for atualizar e volta a paginar quando não for mais necessário.

- **Itere:**

Repita avaliar-ordenar-atualizar até convergir. No fim, α vira o vetor de pesos definitivo do TinyML (grava como $\text{TinyML_model_weights} \leftarrow \alpha$).

Start

Step 1: Let, $w \leftarrow W ol f$.

Step 2: Let, $n = 50w$.

Step 3: For $i = 0$ to n :

$w \leftarrow \text{random weights}$

Step 4: Let, $f_i \leftarrow \text{fitness}(w_i)$

$f \leftarrow \text{sort}(f)$

$\alpha \leftarrow f(0)$, $\beta \leftarrow f(1)$, $\delta \leftarrow f(2)$ and $\omega \leftarrow f(3 \text{ to } n - 1)$

Step 5: SRAM $\leftarrow \alpha$, β and δ .

Swap Space $\leftarrow \omega$.

Step 6: Use α , β and δ to search solution space.

Step 7: SRAM $\leftarrow \omega$

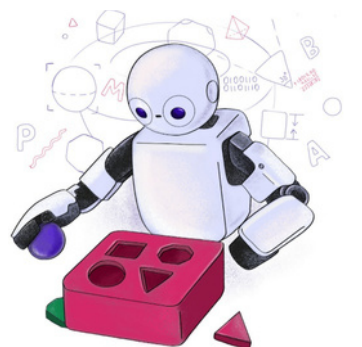
ω updated according to α , β and δ

Step 8: Step 4 to 7 are iterated till convergence.

Step 9: After convergence, $\alpha == \text{solution}$

Step 10: $\text{TinyML_model_weights} \leftarrow \alpha$.

End



Resultados



Os otimizadores por gradiente (RMSProp, Adam etc.) atingem acurácias mais altas (até 0,97), mas com treinos mais lentos e alto custo computacional.

Já os de enxame (GWO, PSO, WOA, etc.) mantêm métricas muito próximas (diferença $< 3\%$) e tempo de treino até 50% menor.

O GWO destaca-se pelo equilíbrio entre precisão, estabilidade e baixo custo, sendo o mais viável para execução TinyML em microcontroladores.

Table 3

Performance of the base neural network with gradient optimizers compared with grey wolf optimizer. The performances of grey wolf optimizer are very close to the gradient optimizers for all test datasets.

		Datasets												
		Heart Attack	Diabetes	Kidney stone	Breast Cancer	BMI Index	Titanic	Fish Species	Wheat seed	Red Wine	Customer behav	Sonar Pred	Loan pred	Airline Pred
RMSProp	Accuracy	0.8	0.82	0.81	0.94	0.85	0.78	0.82	0.87	0.63	0.87	0.85	0.97	0.94
	Precision	0.79	0.82	0.8	0.94	0.85	0.75	0.81	0.88	0.59	0.88	0.83	0.98	0.93
	Recall	0.77	0.8	0.8	0.92	0.84	0.75	0.81	0.87	0.58	0.86	0.83	0.97	0.93
	Train Time	32.4	33.5	46.4	37.8	33.0	36.3	34.7	34.5	33.2	37.5	40.7	41.0	47.9
Adam	Accuracy	0.79	0.83	0.81	0.93	0.83	0.81	0.83	0.87	0.64	0.88	0.84	0.97	0.95
	Precision	0.79	0.8	0.79	0.91	0.84	0.79	0.8	0.87	0.61	0.85	0.8	0.97	0.94
	Recall	0.75	0.78	0.77	0.9	0.82	0.78	0.77	0.86	0.6	0.85	0.81	0.97	0.91
	Train Time (s)	44.0	36.7	50.1	35.0	33.7	34.9	35.7	39.4	33.1	37.5	41.6	38.4	37.5
Adagrad	Accuracy	0.77	0.77	0.79	0.93	0.83	0.78	0.8	0.85	0.61	0.87	0.82	0.97	0.94
	Precision	0.78	0.75	0.8	0.93	0.83	0.77	0.78	0.86	0.57	0.85	0.79	0.98	0.92
	Recall	0.75	0.74	0.79	0.92	0.81	0.75	0.78	0.83	0.57	0.82	0.78	0.97	0.91
	Train Time (s)	33.9	37.5	33.6	36.1	32.2	33.9	35.9	45.7	33.6	38.0	41.8	38.0	37.9
SGD	Accuracy	0.77	0.79	0.8	0.9	0.8	0.8	0.77	0.86	0.6	0.83	0.85	0.95	0.93
	Precision	0.77	0.8	0.8	0.9	0.79	0.79	0.79	0.85	0.59	0.81	0.83	0.94	0.9
	Recall	0.74	0.79	0.77	0.88	0.78	0.79	0.76	0.85	0.58	0.8	0.84	0.92	0.9
	Train Time (s)	33.4	42.4	33.3	36.4	32.8	49.5	34.3	35.2	32.4	39.2	41.6	40.1	52.6
GWO	Accuracy	0.75	0.78	0.78	0.92	0.81	0.79	0.78	0.86	0.58	0.85	0.8	0.97	0.92
	Precision	0.75	0.76	0.78	0.91	0.81	0.77	0.78	0.86	0.55	0.85	0.8	0.97	0.91
	Recall	0.75	0.75	0.77	0.88	0.81	0.78	0.75	0.85	0.55	0.82	0.8	0.85	0.91
	Train Time (s)	17.6	15.8	15.9	18.2	14.79	15.4	17.5	17.8	18.8	14.6	17.8	17.2	18.8

Table 4

Performance of base neural network for swarm optimizers. Similar accuracy with minor variation can be observed for all optimizers for all test datasets.

		Datasets												
		Heart Attack	Diabetes	Kidney stone	Breast Cancer	BMI Index	Titanic	Fish Species	Wheat seed	Red Wine	Customer behav	Sonar Pred	Loan pred	Airline Pred
GWO	Accuracy	0.75	0.78	0.78	0.92	0.81	0.79	0.78	0.86	0.58	0.85	0.8	0.97	0.92
	Precision	0.75	0.76	0.78	0.91	0.81	0.77	0.78	0.86	0.55	0.85	0.8	0.97	0.91
	Recall	0.75	0.75	0.77	0.88	0.81	0.78	0.75	0.85	0.55	0.82	0.8	0.85	0.91
	Train time (s)	17.6	15.8	15.9	18.2	14.7	15.4	17.5	17.8	18.8	14.6	17.8	17.2	18.8
PSO	Accuracy	0.77	0.78	0.79	0.92	0.83	0.78	0.8	0.86	0.61	0.85	0.82	0.97	0.94
	Precision	0.76	0.75	0.77	0.9	0.83	0.76	0.8	0.86	0.59	0.85	0.81	0.97	0.92
	Recall	0.77	0.75	0.78	0.9	0.83	0.77	0.79	0.86	0.58	0.85	0.79	0.97	0.91
	Train time (s)	13.5	14.2	13.9	15.6	13.5	12.2	13.5	12.8	16.4	11.8	12.2	13.9	13.4
WOA	Accuracy	0.77	0.78	0.78	0.91	0.79	0.79	0.78	0.85	0.53	0.85	0.8	0.96	0.92
	Precision	0.77	0.78	0.78	0.9	0.78	0.79	0.78	0.83	0.51	0.83	0.79	0.96	0.92
	Recall	0.75	0.77	0.78	0.9	0.75	0.77	0.78	0.81	0.51	0.81	0.79	0.96	0.9
	Train time (s)	15.7	13.2	13.9	16.3	13.3	13.6	13.6	14.5	16.6	12.2	14.7	14.9	15.5
CSO	Accuracy	0.77	0.77	0.79	0.91	0.8	0.78	0.8	0.85	0.55	0.87	0.82	0.96	0.94
	Precision	0.75	0.77	0.79	0.9	0.79	0.78	0.8	0.83	0.51	0.88	0.81	0.95	0.94
	Recall	0.75	0.75	0.79	0.88	0.77	0.75	0.8	0.82	0.52	0.88	0.8	0.92	0.92
	Train time (s)	17.9	16.6	16.7	18.7	14.7	15.8	17.9	18.3	18.6	14.7	17.9	17.9	18.6
FA	Accuracy	0.75	0.78	0.77	0.88	0.8	0.79	0.78	0.85	0.55	0.85	0.8	0.95	0.92
	Precision	0.73	0.77	0.78	0.85	0.79	0.79	0.78	0.83	0.56	0.85	0.79	0.93	0.92
	Recall	0.73	0.77	0.77	0.83	0.77	0.76	0.78	0.83	0.53	0.85	0.77	0.92	0.91
	Train time (s)	18.4	18.5	19.1	18.7	15.7	16.6	18.3	18.9	19.8	16.7	18.5	18.3	19.2

Resultados



Table 5
Memory profiling of gradient optimizers compared with grey wolf optimizer. Based on all datasets, on average, 71% less memory was required for the grey wolf optimizer. All memory units are in bytes.

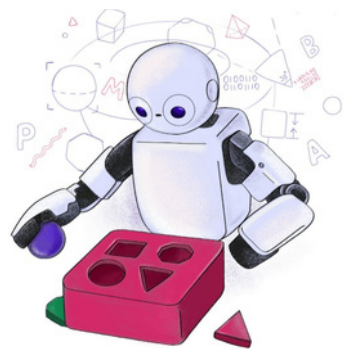
Datasets	RMSProp		Adam		Adagrad		SGD		GWO	
	Average memory	Peak memory	Average memory	Peak memory	Average memory	Peak memory	Average memory	Peak memory	Average memory	Peak memory
Heart Attack	2 242 967	2 945 385	2 467 246	2 642 638	2 452 120	2 943 929	1 532 723	2 326 915	1 104 980	2199612
Diabetes	2 548 475	3 143 130	6 428 518	6 596 054	2 686 408	3 118 222	2 576 836	2 707 160	881 064	6918887
Kidney stone	2 134 071	2 366 619	2 444 938	2 853 278	2 716 102	2 788 268	2 326 971	2 956 792	757 982	890 998
Breast Cancer	2 494 854	2 938 706	2 266 148	2 910 662	2 489 720	2 940 412	2 573 524	2 645 708	584 376	4964840
BMI Index	1 456 154	2 776 145	2 275 425	2 788 455	2 824 774	2 947 259	2 626 626	2 950 570	556 226	3839738
Titanic	2 459 515	2 725 853	2 350 341	3 003 149	2 459 374	2 960 745	4 736 683	4 814 531	586 000	4966904
Fish Species	1 293 301	2 483 333	2 885 492	2 958 869	2 380 704	2 743 866	2 497 125	2 650 605	556 856	3826928
Wheat seed	2 500 126	2 994 810	2 207 895	2 614 542	2 666 117	3 124 743	3 348 989	3 475 938	624 392	5004496
Red Wine	2 668 115	3 016 256	2 495 745	2 832 105	2 540 027	2 832 772	2 084 084	2 920 331	662 196	1501412
Customer behav	2 493 516	3 043 777	2 597 755	3 025 603	2 602 761	2 871 963	2 630 919	2 948 135	447 176	679 584
Sonar Pred	1 764 348	4 961 068	2 575 615	2 917 373	2 307 271	2 746 716	2 619 592	2 691 061	780 443	4271107
Loan pred	1 196 394	2 613 575	2 046 825	2 875 430	1 319 714	2 526 311	2 065 688	2 896 170	476 551	709 487
Airline Pred	2 250 363	2 846 240	1 862 511	2 178 861	2 120 356	2 763 416	2 482 734	2 818 811	1 095 508	2190140

Table 6
Memory profiling of swarm optimizers. Both Grey Wolf and Particle Swarm required very little memory. All memory units are in bytes.

Datasets	GWO		PSO		WOA		CSO		FA	
	Average memory	Peak memory	Average memory	Peak memory	Average memory	Peak memory	Average memory	Peak memory	Average memory	Peak memory
Heart Attack	1 104 980	2 199 612	1 099 796	2 194 484	1 388 067	2 546 568	2 575 573	3 218 389	1 241 805	1859685
Diabetes	881 064	6 918 887	764 242	4 249 394	844 342	4 029 888	1 732 115	4 928 867	769 920	4041569
Kidney stone	757 982	890 998	593 124	984 460	677 142	810 158	1 257 063	1 546 511	429 738	4736618
Breast Cancer	584 376	4 964 840	601 042	4 990 852	611 406	4 822 113	1 251 470	8 711 533	428 236	4752881
BMI Index	556 226	3 839 738	755 072	4 246 536	5 014 248	5 373 952	1 745 748	4 942 700	5 032 086	5397406
Titanic	586 000	4 966 904	720 102	5 101 073	606 023	4 832 326	1 232 630	5 469 926	638 776	8676321
Fish Species	556 856	3 826 928	761 159	4 252 231	5 015 085	5 374 789	1 743 792	4 940 576	4 512 935	5374818
Wheat seed	624 392	5 004 496	624 804	5 427 452	621 314	4 847 664	1 248 681	9 834 103	691 805	4908117
Red Wine	662 196	1 501 412	595 932	987 316	1 136 340	1 869 100	660 584	1 502 120	782 940	1502220
Customer behav	447 176	679 584	591 804	983 244	644 052	841 564	1 397 575	1 558 567	691 363	829 995
Sonar Pred	780 443	4 271 107	754 960	4 246 424	5 029 458	5 389 162	1 745 137	4 942 145	4 527 597	5389413
Loan pred	476 551	709 487	426 352	5 130 671	697 813	850 224	1 264 928	1 554 816	627 968	831 152
Airline Pred	1 095 508	2 190 140	1 117 695	2 212 775	1 256 618	1 868 882	2 344 437	3 228 133	1 255 099	1873035

Os otimizadores por gradiente consomem cerca de 2,4 MB, inviáveis para MCUs de 256 KB. O GWO usa apenas ~0,7 MB, garantindo 71% de economia de memória.

Entre os de enxame, GWO e PSO são os mais leves, mas o GWO se destaca por sua arquitetura eficiente de swap/paging, viabilizando o treinamento direto on-device.



Resultados e Discussões



Por que não gradiente?

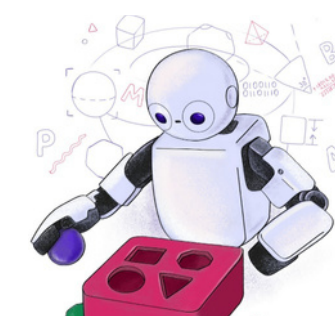
Mesmo quando Adam/RMSProp são ótimos em acurácia, não cabem com folga no envelope de 256 KB sem engenharia agressiva; o regime de buffers de gradientes/momentos pesa em RAM.

Por que swarm?

Meta-heurísticas não armazenam estados complexos por parâmetro — operam com populações e atualizações aritméticas simples; assim, multiplicam a chance de o treino caber na RAM/flash (especialmente com poda de pesos e aproximações de ativação).

Por que GWO (e não PSO)?

Embora PSO tenha ligeira vantagem média de métrica (Tabela 4), o GWO casa melhor com o modelo de memória proposto (manter α , β , δ residentes; ω paginados) e entrega a economia de $\sim 71\%$ vs. gradientes (Tabela 5), garantindo treino on-device com 256 KB sem comprometer demasiadamente as métricas.



Exemplos de aplicação



“mini-projeto” pronto para Google Colab que:

- Carrega o dataset Heart Attack (303 amostras, 13 atributos, target binário)
- Treino: 193 amostras, validação: 49 amostras e teste: 61 amostras
- Constrói o mesmo modelo do artigo (Densa-6 → Densa-20 → Saída, ReLU),
- Treina com Adam,
- Treina o mesmo modelo com Grey Wolf Optimizer (GWO),
- Compara métricas (acc/prec/recall/F1, curva de perda),
- Aplica otimizações do lado GWO inspiradas nos autores (pruning por magnitude + simulação de memória com float16) e reavalia.

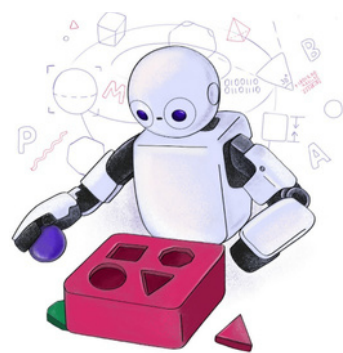
Código disponível em: <https://github.com/Rafa-Laureano/TP558---Topicos-avancados-ML/tree/main/Semin%C3%A1rio%20TinyWolf>

```
class TinyNet(nn.Module):
    def __init__(self, in_dim, hidden1=6, hidden2=20, out_dim=1):
        super().__init__()
        self.fc1 = nn.Linear(in_dim, hidden1)
        self.fc2 = nn.Linear(hidden1, hidden2)
        self.fc3 = nn.Linear(hidden2, out_dim)
    def forward(self, x):
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        return self.fc3(x) # logits

def make_model():
    model = TinyNet(input_dim, 6, 20, 1).to(device)
    return model

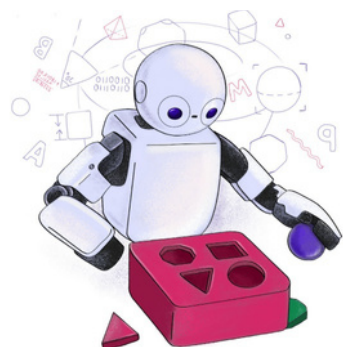
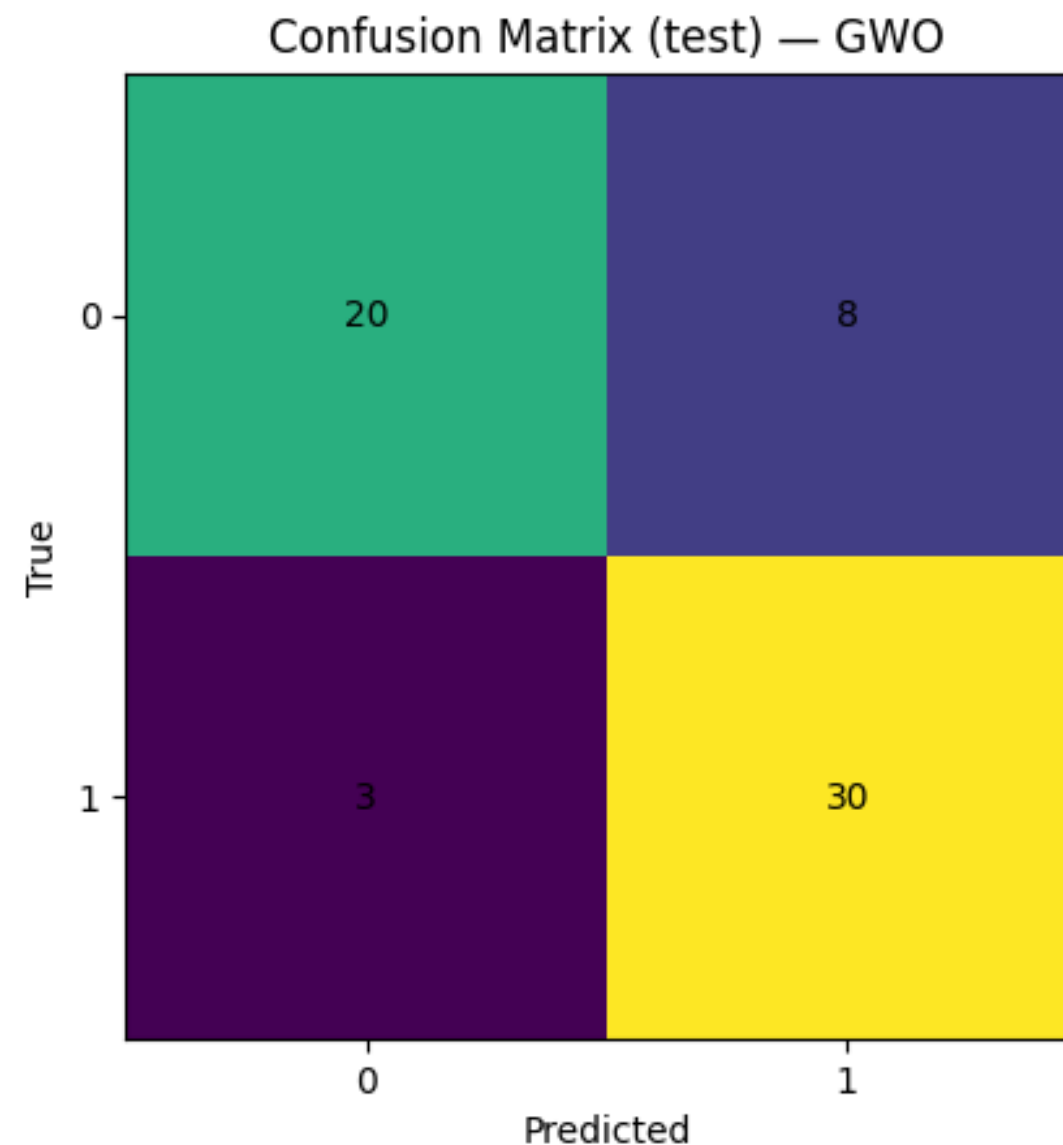
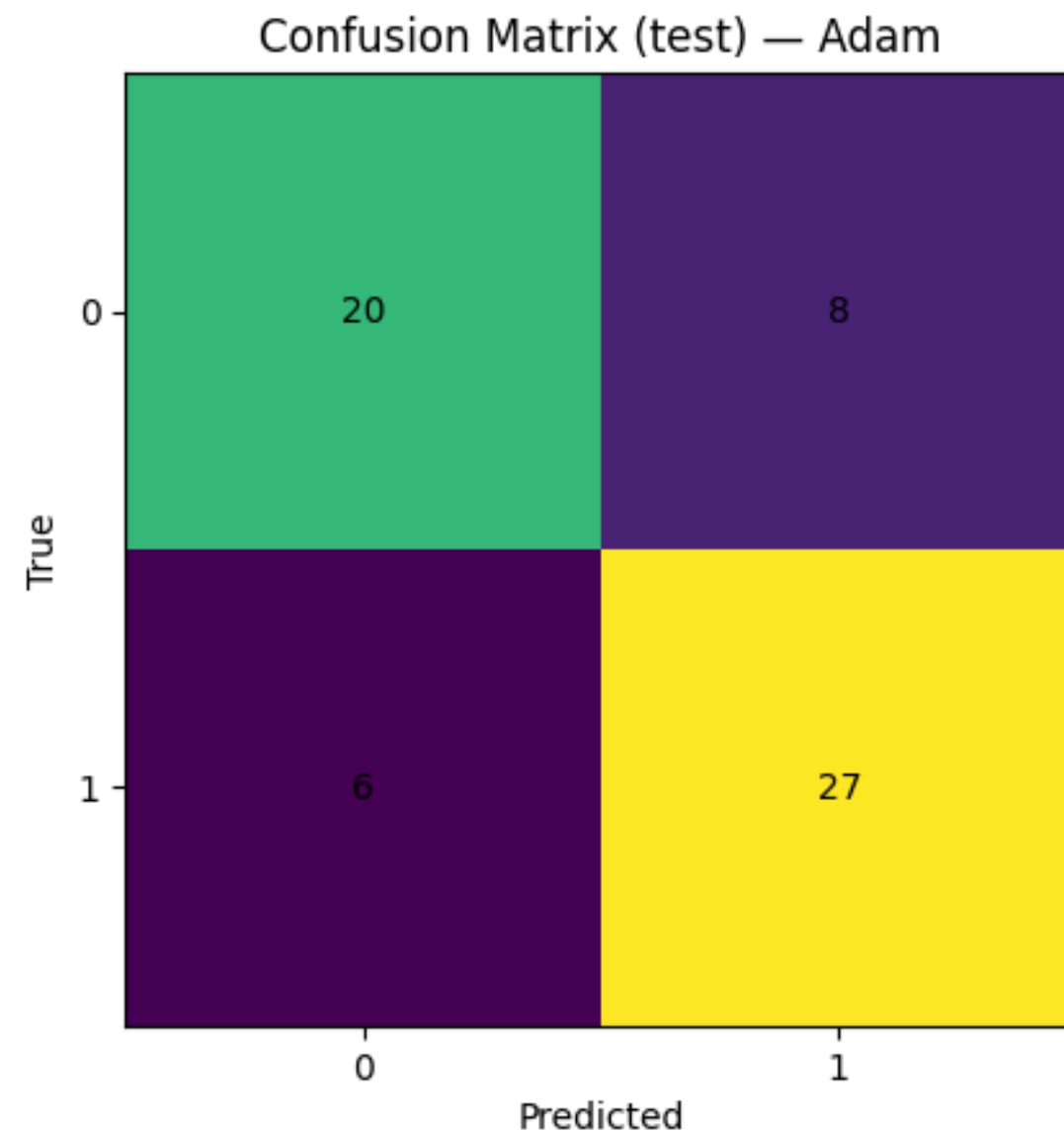
def metrics_from_logits(logits, y_true):
    probs = torch.sigmoid(logits).detach().cpu().numpy().ravel()
    yhat = (probs >= 0.5).astype(int)
    y_true = y_true.detach().cpu().numpy().ravel()
    acc = accuracy_score(y_true, yhat)
    prec = precision_score(y_true, yhat, zero_division=0)
    rec = recall_score(y_true, yhat, zero_division=0)
    f1 = f1_score(y_true, yhat, zero_division=0)
    return acc, prec, rec, f1
```

13 → 6 → 20 → 1 neurônio



Exemplos de aplicação

Ambos têm 20 acertos na classe 0 e 8 erros de falso positivo.
O GWO supera o Adam na classe 1 (30 vs. 27 acertos).
Assim, o GWO apresenta melhor sensibilidade (recall) para a classe 1, sendo mais eficiente em detectar os casos positivos.

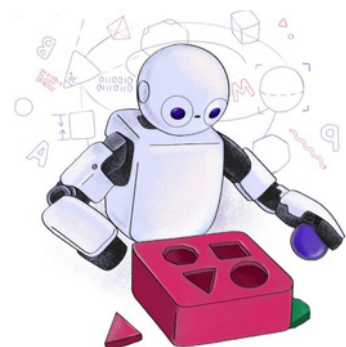
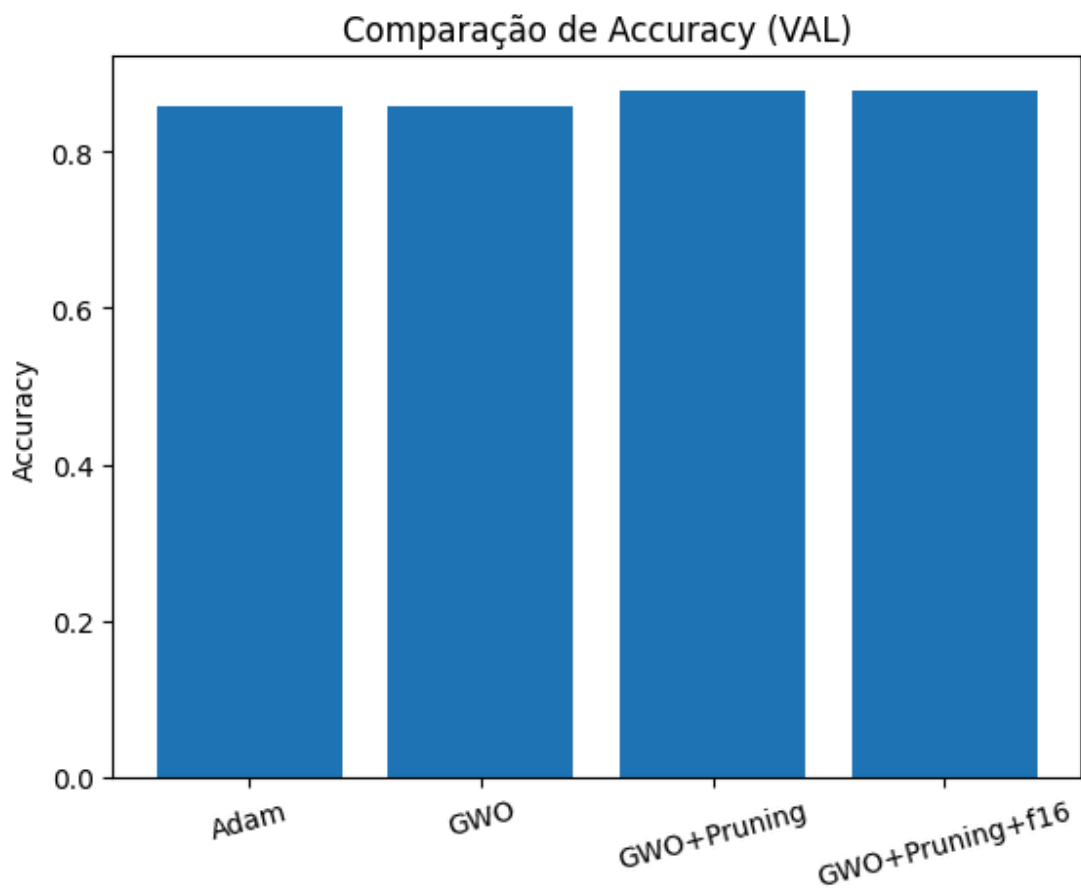
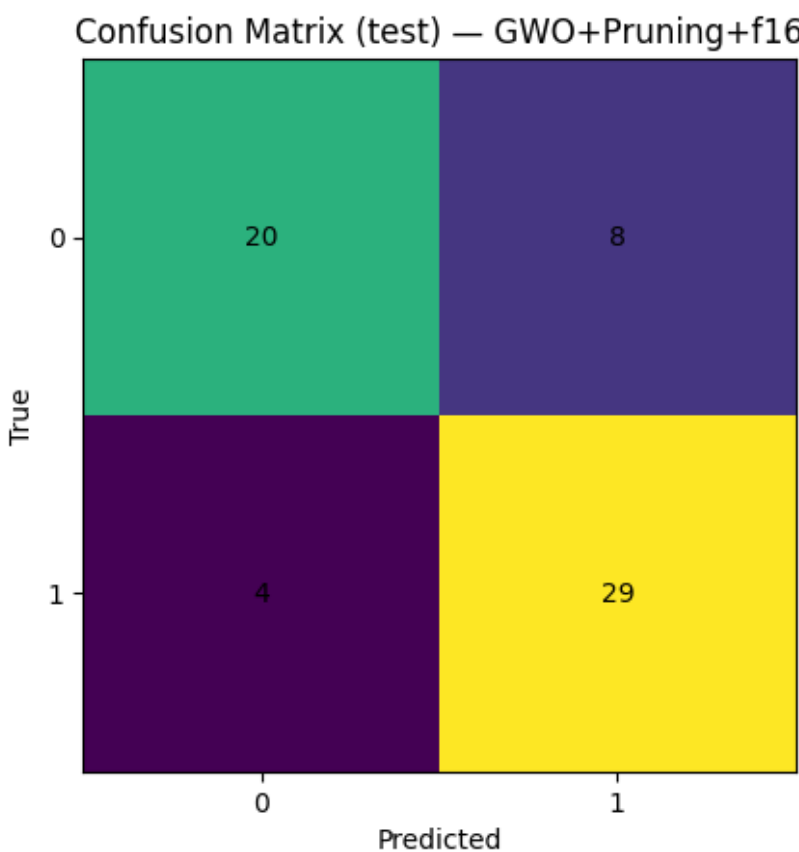
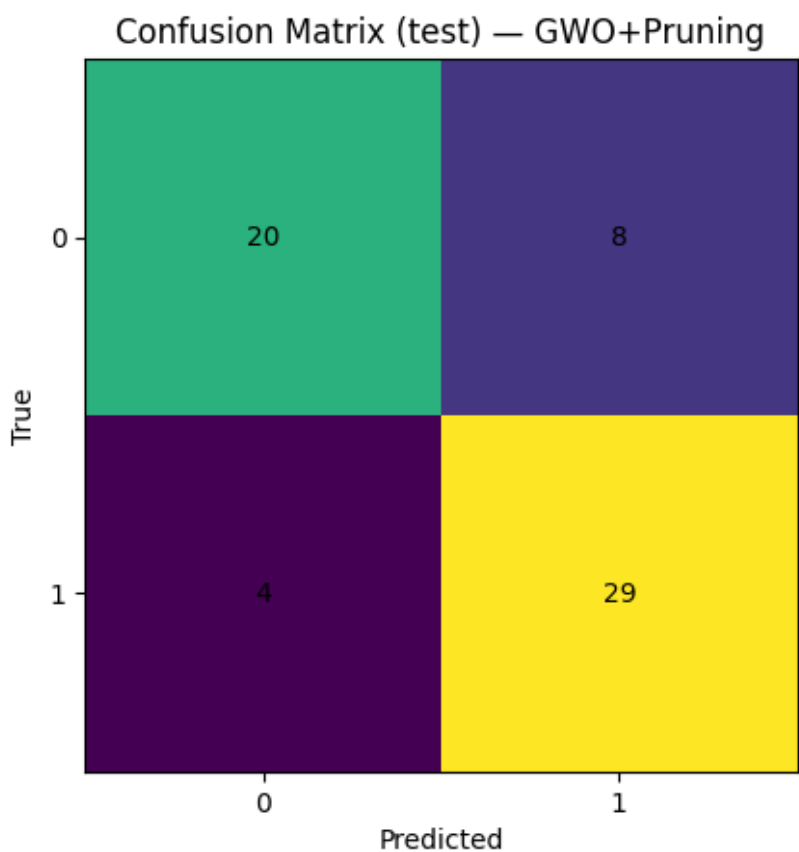
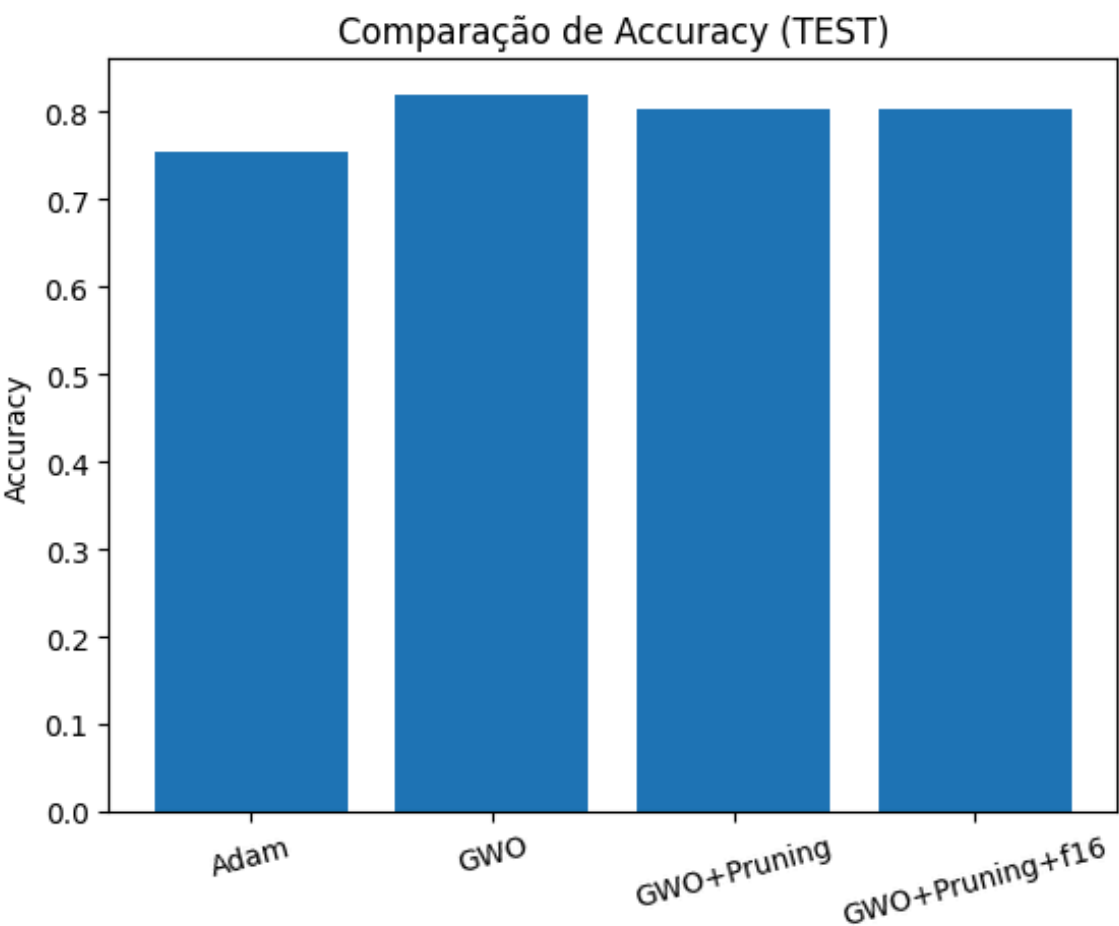


Exemplos de aplicação

Depois o GWO foi combinado com:

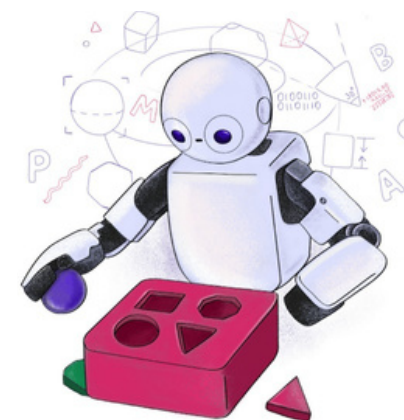
- Pruning → remove 50% dos pesos menos relevantes.
- f16 (half precision) → reduz o tamanho dos pesos de 32 bits para 16 bits (menor memória, igual desempenho).

Modelo	Split	Accuracy	Precision	Recall	F1
Adam	Teste	77	77	82	79
GWO	Teste	82	79	91	85
GWO + Pruning	Teste	80	78	88	83
GWO + Pruning + f16	Teste	80	78	88	83



Perguntas?

Inatel



Referências

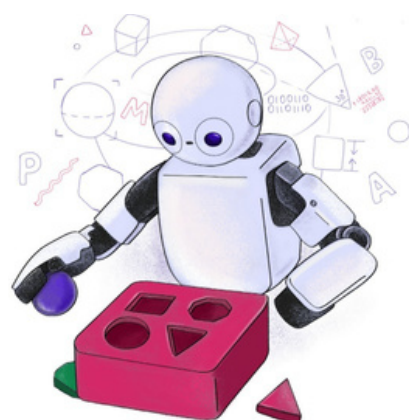
[1] L. Dutta and S. Bharali, “TinyML Meets IoT: A Comprehensive Survey,” Internet of Things, vol. 16, p. 100461, 2021. [Online]. Available: <https://doi.org/10.1016/j.iot.2021.100461> referencia_tinyML

[2] S. Adhikary, S. Dutta, and A. D. Dwivedi, “TinyWolf — Efficient on-device TinyML training for IoT using enhanced Grey Wolf Optimization,” Internet of Things, vol. 28, p. 101365, 2024. [Online]. Available: <https://doi.org/10.1016/j.iot.2024.101365> terceiro_artigo

Github: Available: <https://github.com/Rafa-Laureano/TP558---Topicos-avancados-ML/tree/main/Semin%C3%A1rio%20TinyWolf>

Quizz:

<https://docs.google.com/forms/d/e/1FAIpQLSdDgk00zpWgD5INWnixxUBTbWcAkVQ8LAZBTTFski2v7KjkSw/viewform?usp=header>



Obrigado!

Inatel

